

srcV1 実装補習：仕様書を読んで main・led・timer の関係を理解する

0. この資料の目的

この資料は、srcV1 の答えのコードを示すものではありません。

目的は、仕様書を読んで、次のことを確認することです。

- どのファイルを変更するのか
- どの関数を追加・変更するのか
- main.c , led.c , led.h , timer がどのように関係するのか
- シーケンス図から関数呼び出しをどう読むのか
- 仕様書のどこを見れば、自分の止まっている箇所を解決できるのか

実装で迷ったときは、まずこの資料を見て、**仕様書のどこを読めばよいか**を確認してください。

1. 仕様書全体の地図を見る

まず、仕様書 p.1 の目次を見ます。

今回、実装で特に読むべき場所は次の4か所です。

読む場所	目的
p.2～p.3 仕様V0からV1への主な変更点	何が変わったかを知る
p.17～p.18 呼び出し手順図	main , timer , led の関係を知る
p.19～p.20 関数仕様	led.c / led.h に何を書くかを知る
p.20～p.22 main関数の変更	main.c に何を書くかを知る

注意：

仕様書は上から順番に読むだけでなく、**いま知りたいことに応じて読む場所を選ぶ**ことが大切です。

おすすめの読む順番

1. p.2～p.3 で、srcV0 から srcV1 への変化を確認する
2. p.17～p.18 の呼び出し手順図で、main ・ timer ・ led の関係を見る
3. p.19～p.20 で、led.c / led.h に追加するものを見る
4. p.20～p.22 で、main.c の変更箇所を見る

2. srcV0 から srcV1 で何が変わったか

仕様書 p.2～p.3 を読みます。

注目する箇所

仕様書 p.2 の「LED制御ロジックの再設計」を読みます。

ここでは、次の3点が重要です。

1. V0で使われていた旧 led() 関数は ledout() に名称変更される
2. V1の led() 関数は、点灯パターンの設定関数になる
3. 点滅処理のロジックは ledexec() 関数に集約される

読み替え

srcV0：

main.c の中で、点灯・消灯を判断していた。

srcV1：

main.c は led.c に設定値を渡す。

点灯・消灯の判断は led.c の ledexec() が行う。

ここで理解すること

srcV1では、main.c がすべてを直接行うのではなく、LED点滅に関する仕事を led.c に任せる形に変わります。

main.c

実行時オプションを読む

LEDの設定値を led.c に渡す

ledexec() を定期的と呼ぶ

led.c

LEDの設定値を覚える

timer の値を見て、点灯・消灯を判断する

ledout() を呼んで実際に出力する

確認

Q1. V0の led() は、V1では何という名前に変更されますか。

Q2. V1の led() は、LEDを直接点灯する関数ですか。それとも点灯パターンを設定する関数ですか。

Q3. 点灯・消灯の判断を行う関数は何ですか。

▶ 答え

3. 呼び出し手順図を読む

仕様書 p.17～p.18 の「呼び出し手順図」を見ます。

この図は、LED点滅部分について、**どのモジュールが、どの関数を、どの順番で呼ぶか**を示しています。

シーケンス図の基本的な読み方

図の要素	読み方
縦方向	時間の流れ。上から下へ読む
縦の点線	登場するモジュールやオブジェクト
横矢印	関数呼び出し
細長い四角	そのモジュールの処理が実行されている区間

図の要素	読み方
par	並行して動く処理があることを表す
loop	繰り返し処理を表す
alt	条件分岐を表す

この図に出てくる3つの担当

担当	役割
main	プログラム全体を進める。初期化し、ループを回す
timer	時間を数える
led	LEDの設定を保持し、点灯・消灯を実行する

重要：

- timer は「時間を数える」だけであり、LEDを直接点灯するわけではありません。
- led は「LEDの設定を覚える」だけでなく、 ledexec() により点灯・消灯を実行します。
- main は「設定」と「定期的な呼び出し」を担当します。

4. 初期化部分を読む

仕様書 p.18 の上半分を見ます。

main から led に対して、次の関数呼び出しが行われます。

```
ledblinkmode(led_blink_mode);  
ledblinktime(led_blink_period);
```

これは、main.c で読み取った実行時オプションを、led.c 側に保存する処理です。

次に、main から timer に対して、次のような関数呼び出しが行われます。

```
init_timer0(...);  
start_timer0();
```

これは、`timer` に時間を数えさせるための処理です。

ここで理解すること

初期化部分では、次の2つを行います。

1. LEDの設定値を `led.c` に渡す
2. `timer` を開始する

`main.c` で値を読む

↓

`ledblinkmode()`, `ledblinktime()` で `led.c` に渡す

↓

`timer` を開始する

確認

Q1. 点滅モードを `led.c` に渡す関数は何ですか。

Q2. 点滅周期を `led.c` に渡す関数は何ですか。

Q3. `timer` はLEDを点灯する担当ですか。それとも時間を数える担当ですか。

▶ 答え

5. メインループ部分を読む

仕様書 p.18 の下半分を見ます。

メインループの中で、`main` から `led` に対して次の関数が呼ばれます。

```
led(led_pattern);  
ledexec();
```

この2つは役割が違います。

関数	役割
<code>led(led_pattern)</code>	点灯パターンを <code>led.c</code> に保存する

関数	役割
ledexec()	保存された設定と timer の値を使って、点灯・消灯を判断して実行する

重要：

ledexec() は1回だけ呼ぶ関数ではありません。

while(1) の中で繰り返し呼ばれることで、点滅を実現します。

仕様書 p.21～p.22 との対応

仕様書 p.21～p.22 では、V0のメインループ内の点滅制御部分を削除し、代わりに次の2行を追加すると説明されています。

```
led(led_pattern); /// 点滅パターンの設定
ledexec();          /// 点灯・消灯の判断および実行
```

また、ledexec() は必ずメインループ内で定期的に実行する必要がある、と書かれています。

確認

Q1. led(led_pattern) は、LEDを直接点灯する関数ですか。

Q2. ledexec() は、どこで繰り返し呼び出す必要がありますか。

Q3. ledexec() が呼ばれないと、LEDの点灯・消灯判断は行われますか。

▶ 答え

6. led.h と led.c の違い

仕様書 p.19 の「関数仕様(LEDモジュールの変更)」を読みます。

led.h に書くもの

led.h は、他のファイルから使ってよいものを知らせるためのファイルです。

仕様書では、次のように説明されています。

- 公開関数： led.h にプロトタイプ宣言を追加する

- 公開変数、公開構造体： `led.h` に定義を追加する

今回、 `main.c` から呼ばれる関数は `led.h` に宣言が必要です。

例：

```
void led(int bitPattern);
void ledblinkmode(int mode);
void ledblinktime(int msec);
void ledexec(void);
```

led.c に書くもの

`led.c` は、**LED制御の実際の処理を書くファイル**です。

仕様書では、次のように説明されています。

- 公開関数： `led.c` に定義を追加する
- 非公開関数、非公開変数： `led.c` に `static` 宣言で定義する

つまり、外から使う名前は `led.h` に見せます。

外から直接触らせない変数は `led.c` の中に隠します。

確認

Q1. `main.c` から呼び出す関数のプロトタイプ宣言は、 `led.c` と `led.h` のどちらに書きますか。

Q2. `_ledValue` は、 `main.c` から直接使う変数ですか。

Q3. `static` を付けた変数は、他のファイルから直接使えますか。

▶ 答え

7. 構造体 `ledv` の読み方

仕様書 p.19 の「LEDの点灯に関するデータを扱う構造体(`ledv`)」を読みます。

`ledv` は、LED点滅に必要な設定をまとめた箱です。

```
typedef struct
{
    int pattern; // 点灯パターン
    int period;  // 点滅周期 [msec]
    int mode;    // 点滅/点灯flag 1〜3:点滅, 0:点灯
} ledv;
```

メンバー	何を表すか
pattern	どのLEDを点灯対象にするか
period	点滅周期
mode	点滅モード。0なら常時点灯、1〜3なら点滅

この構造体を使う理由

LED点滅に関係する設定を、ばらばらの変数ではなく、1つのまとまりとして扱うためです。

```
点灯パターン pattern
点滅周期      period
点滅モード    mode
```

これらをまとめて ledv として扱う。

8. ledvp と lv() の読み方

仕様書 p.20 の「構造体 (ledv) へのポインタの型定義 (ledvp)」と「公開関数 ledvp lv(void)」を読みます。

```
typedef ledv *ledvp;
```

これは、

```
ledv *
```

という型に、

ledvp

という別名を付けています。

今回の範囲では、次のように理解すれば十分です。

名前	今回の意味
ledv	LED設定の箱
ledvp	LED設定の箱の場所を表す型
_ledValue	led.c の中にあるLED設定の実体
lv()	_ledValue の場所を返す関数

仕様書では、lv() は次のように定義されています。

```
ledvp lv(void)
{
    return &_ledValue;
}
```

注意：

ポインタを完全に理解していなくても、今回の実装では、まず **_ledValue の場所を返している** と読めればよいです。

9. ledexec() の読み方

仕様書 p.20 の「公開関数 void ledexec(void)」を読みます。

ledexec() は、LEDを今点けるか、消すかを判断して実行する関数です。

ledexec() が見るもの

ledexec() は、led.c の中に保存されている次の値を見ます。

- _ledValue.pattern
- _ledValue.period

- `_ledValue.mode`

さらに、`timer` の値を見て、時間が経過したかを判断します。

mode が 0 の場合

常時点灯します。

つまり、次のように出力します。

```
ledout(_ledValue.pattern);
```

mode が 1 の場合

点滅します。

周期の半分ごとに、次の2つを切り替えます。

- 点灯する
- 消灯する

点灯するとき：

```
ledout(_ledValue.pattern);
```

消灯するとき：

```
ledout(0);
```

重要

`ledexec()` は、点滅パターンや周期を設定する関数ではありません。
設定済みの値を見て、点灯・消灯を実行する関数です。

10. LED関係の関数の役割まとめ

関数	役割	設定する/実行する
led(int bitPattern)	点灯パターンを保存する	設定する
ledblinkmode(int mode)	点滅モードを保存する	設定する
ledblinktime(int msec)	点滅周期を保存する	設定する
ledexec(void)	点灯・消灯を判断して実行する	実行する
ledout(int bitPattern)	LEDに実際に出力する	実行する

よくある混乱

混乱	正しい理解
led() がLEDを点灯する	V1の led() は点灯パターンを保存する
ledexec() を1回呼べば点滅する	while(1) の中で定期的に呼ぶ必要がある
timer がLEDを点灯する	timer は時間を数えるだけ
_ledValue を main.c から直接変更する	ledblinkmode() などの関数を通して変更する
led.h に処理を書く	led.h には宣言を書く。処理は led.c に書く

11. main.c の変更箇所を読む

仕様書 p.20～p.22 の「main関数の変更」を読みます。

main.c で行う変更は、大きく4つあります。

1. ローカル変数の追加

仕様書 p.21 に、次の3つの変数が示されています。

```
int led_blink_period = 1000; /// 点滅周期[msec]。初期値1000[msec]
int led_blink_mode = 1;      /// 点滅モード。初期値モード1
int led_pattern = 0x05;      /// 点滅パターン。初期値xxxxx0x0
```

これらは、実行時オプションの値を一度 main.c 側で受け取るための変数です。

2. 実行時引数の読み込み

-B, -F, -N の値を getint 関数で読み込みます。

例：

```
prog.elf -B500 -F1 -N0xCC
```

このとき、

- led_blink_period に 500
- led_blink_mode に 1
- led_pattern に 0xCC

が入ります。

3. 初期化コードの追加

while(1) の前に、次の関数呼び出しを追加します。

```
ledblinktime(led_blink_period);
ledblinkmode(led_blink_mode);
```

これは、main.c で読み取った設定値を led.c に渡す処理です。

4. メインループ内の変更

V0の点滅制御処理を削除し、次を追加します。

```
led(led_pattern);
ledexec();
```

- led(led_pattern) は、点灯パターンを led.c に渡します。

- `ledexec()` は、点灯・消灯を判断して実行します。

12. どこで止まっているか確認する

次の表で、自分がどこで止まっているか確認してください。

状態	読む場所
V0とV1の違いが分からない	仕様書 p.2～p.3
<code>main</code> , <code>led</code> , <code>timer</code> の関係が分からない	仕様書 p.17～p.18
<code>led.h</code> に何を書くか分からない	仕様書 p.19
<code>led.c</code> に何を書くか分からない	仕様書 p.19～p.20
構造体 <code>ledv</code> が分からない	仕様書 p.19
<code>ledvp</code> / <code>lv()</code> が分からない	仕様書 p.20
<code>ledexec()</code> が分からない	仕様書 p.20
<code>main.c</code> の変更箇所が分からない	仕様書 p.20～p.22
<code>usage</code> 表示が分からない	仕様書 p.5, p.22
MESA-INTEL の warning が出た	まず、コンパイルエラーか警告かを確認する
赤い波線が出る	まず <code>make</code> の結果を見る

13. 質問する前に確認すること

質問する前に、次の4つを確認してください。

1. どのファイルで止まっているか

- `main.c`
- `led.c`
- `led.h`
- `timer`関係
- `Makefile`

- シミュレータ
2. どの段階で止まっているか
 - 仕様書の意味が分からない
 - シーケンス図の意味が分からない
 - C言語に変換できない
 - ビルドエラーが出る
 - 実行できるが動作がおかしい
 3. 仕様書の何ページを見たか
 4. エラーメッセージがある場合、最初のエラーを1つ書く

質問メモ

止まっているファイル：

止まっている関数：

見た仕様書のページ：

分からないこと：

最初のエラーメッセージ：

14. 授業中の進め方

今日の授業では、進度に応じて2つのグループに分かれます。

検証組

実装が一通り終わった人は、境界値分析とオシロスコープ測定を含む検証に進みます。

ただし、**完成したと思っても、仕様を満たしているかは別**です。

仕様書を見ながら、どの条件を確認すべきか整理してください。

実装補習組

srcV1の実装で止まっている人は、この資料を使って、仕様書の読み方を確認します。

特に、次のことが不安な人は補習に参加してください。

- `main.c` , `led.c` , `led.h` , `timer` の関係が分からない
- シーケンス図の読み方が分からない
- `ledv` , `ledvp` , `lv()` が分からない

- `ledexec()` が何をする関数か分からない
- `main.c` のどこに何を追加すればよいか分からない

補習では、答えのコードを配るのではなく、**仕様書のどこを読めば実装できるか**を確認します。

15. 最後に

srcV1では、C言語の文法だけでなく、次の力が必要になります。

- 仕様書を読む力
- 図から関数呼び出しを読む力
- ファイルごとの役割を区別する力
- 設定する関数と実行する関数を区別する力
- エラーが出たときに、最初のエラーから確認する力

分からないところがあるのは自然です。

ただし、「全部分からない」で止まるのではなく、まずは次のように分けて考えてください。

どのファイルか

どの関数か

仕様書の何ページか

何が分からないのか

そこまで分けられれば、質問しやすくなります。